



Colaboración entre objetos: pasar referencias y compartir estado

El problema (son dos)

La Billetera de la clase pasada quedó bien protegida: nace válida, rechaza los datos que rompen su promesa y no hay forma de dejarla en negativo. Pero está sola. Un pago de verdad no es un número que baja: es plata que sale de una billetera y entra en otra. Para modelar eso hacen falta dos objetos trabajando juntos, y aparece una pregunta que hasta ahora no tuvimos que hacernos: ¿cómo hace un objeto para pedirle algo a otro?

El segundo problema es más raro. Una facultad asigna a cada estudiante un legajo único y correlativo: 10001, 10002, 10003. El legajo de cada estudiante es claramente un dato del estudiante: cada uno tiene el suyo. Pero el próximo legajo a asignar, ¿de quién es? No es de ningún estudiante en particular: es un dato de todos a la vez. Hasta hoy, cada atributo que escribimos vive adentro de un objeto. Este no tiene dónde vivir.

Esta clase responde las dos preguntas, y las respuestas están conectadas: colaborar es pasarse referencias, y lo que no es de ningún objeto en particular vive en la clase.

Colaborar es mandar mensajes

Un programa orientado a objetos funciona como una comunidad: objetos que se piden cosas entre sí. Ese pedido se llama mensaje, y en Java mandar un mensaje es invocar un método público de otro objeto. La sintaxis no tiene nada nuevo: es lo que venimos haciendo cada vez que el programa de prueba escribe `billetera.pagar(1000)`. Lo nuevo de hoy es que el que manda el mensaje puede ser otro objeto.

Para pedirle algo a un objeto hay que tener cómo encontrarlo: una referencia. Y la forma más común de conseguir la referencia de otro objeto es que llegue por parámetro. Así queda una transferencia, escrita adentro de `Billetera`:

```
public boolean transferir(Billetera destino, double monto) {
    if (destino == null) {
        return false;
    }
    if (!this.pagar(monto)) {
        return false;    // sin plata: la transferencia no ocurre
    }
    destino.cargar(monto);
    return true;
}
```



Vale la pena detenerse en lo que este método no hace. No toca `destino.saldo`: no puede, es `private`. Le manda el mensaje `cargar`, y la billetera `destino` aplica sus propias reglas para decidir si acepta. La colaboración no rompe el encapsulamiento: lo usa. Cada objeto sigue siendo el único responsable de su estado; lo que cambia es quién le pide las cosas.

También reaparece `this` con su significado completo: adentro de `transferir` hay dos billeteras en juego, `this` (la que paga) y `destino` (la que cobra), y el código necesita nombrarlas para no confundirlas.

Qué viaja en el parámetro

Cuando el `main` ejecuta `miBilletera.transferir(tuBilletera, 1000)`, ¿qué recibe exactamente el parámetro `destino`? ¿Una copia de la billetera? ¿La billetera "original"?

Java tiene una sola regla, sin excepciones: los parámetros se pasan por valor. El método siempre recibe una copia de lo que se le pasa. Lo que cambia según el tipo es qué es ese valor:

1. Con un tipo primitivo (`int`, `double`, `boolean`, `char`), el valor es el dato mismo. El método recibe una copia del número, y lo que haga con esa copia no afecta a la variable original.
2. Con un objeto, el valor de la variable no es el objeto: es una referencia, la dirección donde el objeto vive en la memoria. Lo que se copia es esa referencia. El resultado son dos variables distintas, la de quien llama y el parámetro, apuntando al mismo y único objeto.

Por eso `destino.cargar(monto)` modifica la billetera del `main`: no hay dos billeteras, hay una sola con dos nombres. Y por eso la respuesta canónica, que es pregunta clásica de final: Java pasa siempre por valor; en el caso de los objetos, lo que se copia es el valor de la referencia. Decir que "los objetos se pasan por referencia" es casi correcto, y por eso es tan común. El matiz se ve con una prueba: si adentro del método se escribe `destino = new Billetera(...)`, la variable del `main` sigue apuntando a la billetera de siempre. El método puede usar la referencia recibida para modificar el objeto, pero no puede cambiar a qué objeto apunta la variable de quien lo llamó.

Un dato que no es de nadie: static

Queda el problema del legajo. Todos los atributos que escribimos hasta hoy son miembros de instancia: cada objeto tiene su propia copia. Si el contador `proximoLegajo` se declara así, cada estudiante nace con su propio contador, todos los contadores valen 10001 y todos los legajos salen iguales. El código compila y corre: simplemente hace otra cosa que la que el sistema necesita. En la actividad de esta clase ese error se ve en vivo.

La palabra clave `static` declara un miembro de clase: una única copia que vive en la clase y es compartida por todos sus objetos.



```
public class Estudiante {  
  
    private static int proximoLegajo = 10001; // uno solo, de la clase  
    private final int legajo;                // uno por estudiante  
  
    public Estudiante(String nombre, String apellido) {  
        // ... validaciones ...  
        this.legajo = proximoLegajo;  
        proximoLegajo++;  
    }  
}
```

Cada estudiante nuevo toma el valor actual del contador compartido y lo incrementa para el siguiente, sea del curso que sea y se cree donde se cree. Un solo contador, una sola numeración.

Los métodos también pueden ser `static`: son métodos de la clase, que se ejecutan sin ningún objeto. Adentro de un método `static` no existe `this` (¿sobre qué objeto estaría parado?), y por eso no puede acceder directamente a miembros de instancia. Ya usamos varios sin nombrarlos: `Math.PI` es un atributo `static`, `Integer.parseInt(...)` es un método `static`, y el propio `main` de cada programa es `static` porque cuando arranca el programa todavía no existe ningún objeto.

El acceso también dice de quién es el dato: los miembros de clase se acceden con el nombre de la clase, `Estudiante.getProximoLegajo()`, `Math.PI`. Java permite accederlos a través de un objeto (`e1.getProximoLegajo()` compila), pero es mala práctica: el que lee cree que está consultando un dato de `e1`, y en realidad está consultando un dato de toda la clase.

Cuándo usar `static` (y cuándo no)

Los usos legítimos son pocos y claros: contadores y numeradores compartidos como el del legajo, constantes (`Math.PI`, `UN_MAX_INSCRIPCIONES`), y métodos de utilidad que no dependen del estado de ningún objeto (`Integer.parseInt()`).

Fuera de esos casos, `static` merece desconfianza. Un atributo `static` es estado global: cualquier parte del programa puede modificarlo, y cuando algo sale mal no hay forma de saber quién fue, porque el dato no tiene dueño. Es exactamente lo contrario de lo que construimos la clase pasada, donde cada objeto era el único responsable de su estado.

La regla práctica invierte la carga de la prueba: si un dato es de un objeto, va en el objeto. `static` es la excepción que hay que poder justificar, no una opción más.



Característica	Miembros de instancia	Miembros de clase (static)
Pertenencia	De cada objeto: cada instancia tiene su propia copia.	De la clase: una única copia compartida por todas las instancias.
Analogía	El legajo de un estudiante: cada uno tiene el suyo.	El nombre de la universidad: el mismo para todos.
Estado que modela	Individual: el saldo de una billetera, los inscriptos de un curso.	Compartido: el próximo legajo a asignar, una constante como Math.PI.
this	Disponible: los métodos de instancia operan sobre un objeto concreto.	No existe: un método static no está parado sobre ningún objeto.
Acceso	objeto.miembro	NombreClase.miembro (escribir objeto.miembroEstatico compila, pero es mala práctica).

Para pensar antes de la clase

Con la Billetera de la clase pasada a mano, tres preguntas para traer pensadas:

3. En transferir, ¿qué recibe exactamente el parámetro destino? Si el método hiciera `destino = new Billetera(...)`, ¿la billetera del main cambiaría? ¿Y con `destino.cargar(100)`? ¿Por qué una cosa sí y la otra no?
4. Si cada billetera tuviera que tener un número de cuenta correlativo (la primera el 1, la segunda el 2), ¿qué habría que agregarle a la clase? ¿Qué parte sería static y qué parte no?
5. ¿Qué pasaría si por error saldo se declarara static? ¿Qué imprimiría el programa de prueba de la clase pasada? Pensarlo primero; probarlo después cuesta una palabra.

En el proyecto integrador las clases del dominio ya no van a estar solas: se van a guardar unas a otras en listas, se van a recibir por parámetro y van a repartirse responsabilidades. Es exactamente el contenido de esta clase.

Bibliografía sugerida para profundizar

6. Deitel, P., & Deitel, H. (2017). Java How to Program, Early Objects.
7. Capítulo 6: Methods: A Deeper Look. Métodos static y el paso de argumentos, con la discusión de pass-by-value contra pass-by-reference.
8. Capítulo 8: Classes and Objects: A Deeper Look. La sección de static class members es el tratamiento completo del tema.
9. Azpitarte, R. et al. (2009). Introducción a la Programación Orientada a Objetos con Java.
10. Capítulo 3, sección 3.2.4: objetos como argumentos de un método; sección 3.3: miembros de instancia y de clase.



11. Aguilar, L. J. (2011). Programación en Java. McGraw-Hill.
12. Capítulo 8, sección 8.9: miembros static de una clase. Capítulo 9, sección 9.5: paso de argumentos a métodos.